Department of Information Technology

Subject Code: **ITSBL603**

Subject: **DEVOPS LABORATORY**

Course type: SBL

Terms Offered :**Third Year Semester-VI**

Marks/Credits Assigned :100/**02**

Sub Teacher: Mrs.Supriya P.Joshi

Module 1
Introduction to DevOps and Version Control

Contents:

- Definition of DevOps
- Challenges of traditional IT systems & processes
- History and emergence of DevOps
- DevOps definition and principles governing DevOps
- DevOps Lifecycle.
- Basic Operations in a VCS.
- Features and benefits of GIT.

Challenges of traditional IT systems & processes

Traditional IT systems face challenges including :

- poor performance due to slow storage and data processing
- lack of flexibility and integration with modern systems, and significant security vulnerabilities that can lead to data loss and compliance issues.
- They also suffer from data redundancy, inconsistency, and high costs associated with manual processes and hardware maintenance, which hinder scalability and agility.

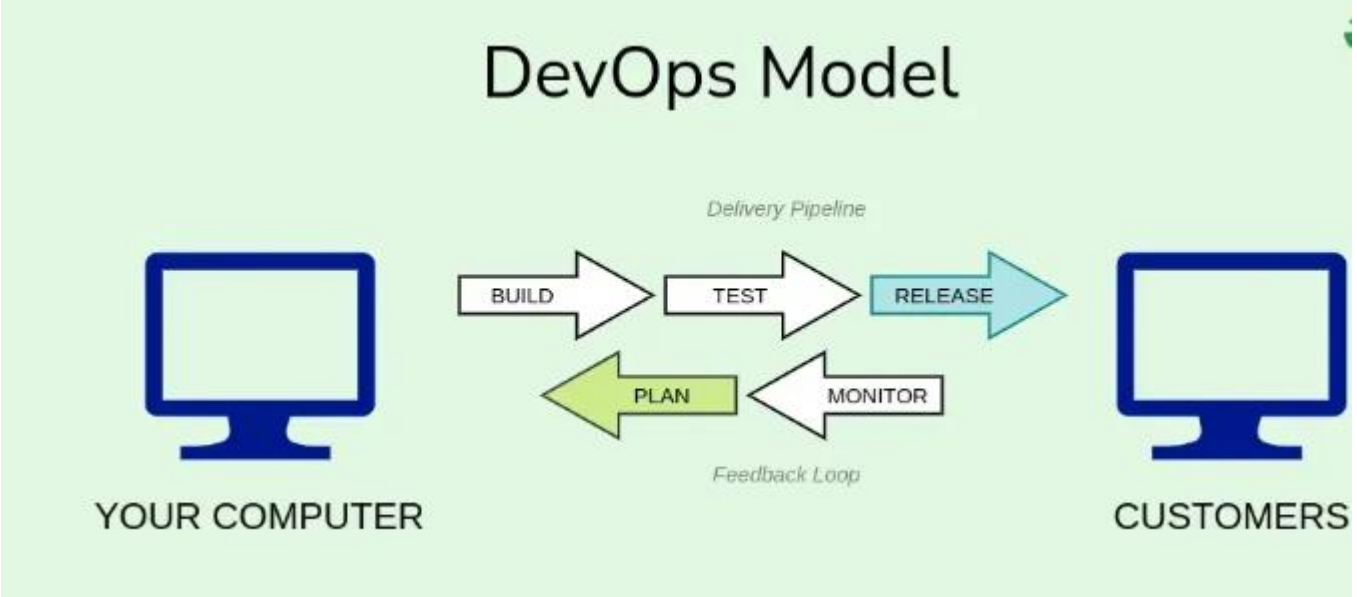
History and emergence of DevOps

- DevOps emerged in the late 2000s as a response to the growing gap between software development and IT operations teams, driven by the rise of Agile methodologies and the need for faster, more collaborative software delivery.
- The term "DevOps" was coined by Patrick Debois in 2009 when he organized the first DevOpsDays conference in Belgium, formalizing a movement to break down silos and automate processes.
- Early milestones included a 2008 "Agile Infrastructure" session and a 2009 Velocity Conference presentation by John Allspaw and Paul Hammond on "10+ Deploys per Day," which highlighted the potential for high-frequency releases.

Definition of DevOps

DevOps is all about automating and streamlining the software development lifecycle so that code moves from development to production quickly, reliably, and securely.

- Here is how the DevOps model flow works:



Stages of DevOps are:

Build Stage

1. Developers write and organize code, using version control tools like Git to track changes.

2. The system automatically compiles and packages the code into a deployable format.

3. Dependencies (external libraries and tools) are included to ensure smooth operation.

4. Common Tools: Git, Jenkins, GitLab CI/CD, Gradle, Maven.

Stages of DevOps are:

Test Stage

1. The software undergoes thorough testing to catch bugs and security risks before release.

2. Different testing methods include:

Unit Testing: Checks individual pieces of code.

Integration Testing: Ensures different parts of the system work together.

Performance Testing: Measures speed and scalability.

Security Testing: Identifies potential vulnerabilities.

3. Automated tests help ensure the software is stable before moving forward.

4. Common Tools: Selenium, JUnit, TestNG, SonarQube.

Stages of DevOps are:

Release Stage

1. The software is deployed in a staging environment to simulate real-world conditions.

2. If everything checks out, the software is rolled out to production using deployment strategies like:

Blue-Green Deployment: Two identical environments switch traffic for a seamless update. "blue" (live) and "green" (new), to minimize risk, allowing seamless traffic switching to the tested new version (green) after deployment, with an instant rollback option to the old version (blue) if issues arise. This approach supports continuous delivery by ensuring users experience no interruption while new features are validated in a production-like setting

Canary Deployment: A small percentage of users get the new version first, ensuring safety.

Rolling Updates: The update is gradually pushed out to all users.

3. Common Tools: Docker, Kubernetes, Ansible, Helm, ArgoCD.

Stages of DevOps are:

Continuous Feedback Loop

- Monitoring & Logging: Track system performance and detect errors.
- User Feedback: Gather insights from customers to enhance features.
- Incident Response: Alert systems notify teams of failures for quick fixes.
- Process Improvement: Teams analyze past releases to optimize automation and workflow.
- Common Tools: Prometheus, Grafana, ELK Stack, Datadog, New Relic.

Principles governing DevOps

Automation: Automating manual and repetitive tasks throughout the software development lifecycle, such as code integration, testing, and deployment, to increase efficiency and reduce errors.

Continuous Integration (CI): Frequently merging code changes from multiple developers into a central repository and verifying them with automated builds and tests to detect bugs early.

Continuous Delivery (CD): Extending CI by automatically deploying all code changes to a testing and/or production environment after the build stage, ensuring that software can be released reliably at any time.

Continuous Feedback: Gathering user feedback at every stage of the lifecycle to ensure that the product aligns with customer needs and to guide continuous improvement.

Principles governing DevOps

Continuous Monitoring: Tracking the performance and behavior of the application in real-time to proactively identify and address potential issues before they escalate.

Measurement: Using metrics to gauge the success of a product and the effectiveness of processes to identify areas for improvement.

Lean Principles: Streamlining the development process by removing unnecessary work and focusing on delivering value efficiently.

DevOps Lifecycle

7 Cs of DevOps

Phases of DevOps Lifecycle

- Plan: This phase focuses on understanding the business needs and gathering feedback from end-users. Teams create a plan that aligns the project with business goals and ensures the right results are delivered.
- Code: In this phase, developers write the actual code for the software. Tools like Git help manage the code, making sure that the code is well-organized and free from security issues or bad coding practices.
- Build: Once the code is written, it is submitted to a central system using tools like Jenkins. This step ensures the code is compiled, and all components are integrated together smoothly.
- Test: The software is then tested to ensure it works properly. This includes different types of tests like security, performance, and user acceptance. Tools like JUnit and Selenium are used to automate these tests and verify the software’s integrity.
- Release: After testing, the software is ready to be released to production. The DevOps team ensures that all checks are passed and then sends the latest version to the production environment.

Phases of DevOps Lifecycle

- Deploy: Using Infrastructure-as-Code (IaC) tools like Terraform, the necessary infrastructure (servers, networks, etc.) is automatically created. Once the infrastructure is set up, the code is deployed to various environments in an automated and repeatable way.
- Operate: Once deployed, the software is available for users. Tools like Chef help manage the configuration and ongoing deployment of the system to ensure it operates smoothly.
- Monitor: This phase involves observing how the software is performing in the real world. Data about user behaviour and application performance is collected to identify any issues or bottlenecks. By monitoring the system, the team can quickly spot and fix problems that may affect performance.

1. Continuous Development

Example:

- Imagine a team building a food delivery app. Instead of waiting to finish the whole app and testing it later, the team adds features one by one:
- On Monday, they add a "Login" feature and test it immediately.
- On Tuesday, they add the "Search for restaurants" feature and test that too.
- Each feature is checked and added to the live app as soon as it's ready. This way, if there's a problem in the "Login" part, they can fix it right away without affecting other parts.

Continuous Integration

Example:

Let's say your team adds a new feature: "Track Delivery Person on Map."

- Developer XYZ writes code for the tracking feature and pushes it to GitHub.
- As soon as the code is pushed, Jenkins picks it up and uses Maven to build the app and test it using JUnit.
- The code goes through SonarQube, which finds a few duplicate lines and suggests better practices.
- Once everything is okay, the final app version (with the new feature) is saved in Nexus as a .jar file.
- This way, every small change is tested, verified, and saved automatically without manual effort.

Continuous Testing

Example:

- Let’s say your team adds a new feature: "Apply Coupon at Checkout."

- After the developer pushes this new feature to GitHub, Jenkins automatically starts the testing process. Tools like Selenium or Testsigma test:

- Does the coupon code apply correctly?
- Does the final price update as expected?
- Does the checkout process still work?

If a problem is found, for example, the app crashes when a wrong coupon is entered the test will fail, and the developer will be notified immediately to fix it. This way, the team avoids pushing broken features to production and ensures the app remains reliable.

4. Continuous Deployment/ Continuous Delivery

Example:

- Suppose the team adds a “Refer & Earn” feature.

- The code is developed, tested, and marked as ready to go live.
- However, the product team decides to launch it during a weekend campaign.
- Until then, the feature stays on standby in the staging area.
- Once approved, the code is manually deployed to production using a single click.

- So, Continuous Delivery ensures every update is deployable anytime, but the actual release can be controlled.

5. Continuous Monitoring

- Let's say your app suddenly takes longer to load the “Order History” page.

- Prometheus tracks this slow response time and sends an alert to the team.
- Grafana shows a graph that spikes during dinner hours when traffic is high.
- The team uses this data to adjust the server settings or optimize the code.
- This prevents crashes and keeps the app smooth for all users, especially during peak hours like dinner time.

6. Continuous Feedback

Example:

- Suppose users complain that the live delivery tracking is not updating fast enough.

- The feedback is collected via app reviews, customer support, or feedback forms.
- The DevOps team analyzes the issue and works with developers to improve the tracking speed.
- The next update includes a fix, and the user experience improves.
- By responding to feedback, the app becomes more reliable and enjoyable to use.

7. Continuous Operations

Example:

Let’s say you need to update the payment system.

Instead of shutting the app down, Continuous Operations ensures the update happens in the background.

Users continue ordering food while the change is made without even noticing.

This keeps customers happy and business running 24/7, especially during peak times like lunch and dinner.

Popular DevOps Lifecycle Tools

1. Plan

Tools: Jira, Trello, Asana

Used for task planning, assignment, and progress tracking.

2. Develop

Tools: Git, GitHub, GitLab, Bitbucket

Enables version control, code collaboration, and branching.

3. Build

Tools: Jenkins, Maven, Gradle

Automates the process of compiling code and managing dependencies.

4. Test

Tools: Selenium, JUnit, TestNG, SonarQube

Conducts automated testing for bugs, code quality, and security vulnerabilities.

Popular DevOps Lifecycle Tools

5. Release/Deploy

Tools: ArgoCD, GitLab CI/CD, AWS CodeDeploy, Azure DevOps, Spinnaker, Terraform

Automates deployment pipelines and software releases.

6. Operate

Tools: Terraform, Ansible, Puppet, Chef

Handles infrastructure provisioning and configuration management.

7. Monitor

Tools: Prometheus, Grafana, ELK Stack, Datadog

Tracks performance, logs, metrics, and system health.

Basic Operations in a VCS.

- The basic operations in a Version Control System (VCS) revolve around tracking changes, collaborating with others, and managing different versions of files. The core operations can be categorized into local actions (working with your own copy) and remote actions (interacting with a shared, central repository).

Basic Operations in a VCS.

Local Operations

These actions manage changes in your local working environment and repository.

- Initialize (init): Sets up a new VCS repository in a project directory, allowing it to start tracking file changes.
- Add (add): Stages modified or new files, marking them to be included in the next commit. This moves changes from the working directory to the staging area.
- Commit (commit): Permanently saves the staged changes into the local repository with a unique identifier (hash) and a descriptive message. This creates a new, permanent version (or "revision") of the project's state.
- Status (status): Displays the current state of the working directory and staging area, showing which files are modified, new, or staged.
- Diff (diff): Shows the differences between various versions of files, such as between the working directory and the last commit, or between two different commits.
- Revert/Reset (revert/reset): Undoes changes. Revert creates a new commit that undoes the changes of a previous commit, while reset moves the current branch pointer to a different commit, discarding subsequent changes (depending on the type of reset).
- Branch: Creates an independent line of development, allowing developers to work on new features or bug fixes without affecting the main codebase.

Basic Operations in a VCS.

Remote Operations

These actions facilitate collaboration and synchronization between local and remote repositories (e.g., on platforms like GitHub or a central server).

Clone (clone): Creates a complete copy of a remote repository, including the entire project history, on a local machine.

Pull (pull): Fetches changes from the remote repository and automatically merges them into the local working copy. This is typically a combination of fetch and merge or rebase.

Push (push): Uploads committed changes from the local repository to the remote repository, making them available to other team members.

Fetch (fetch): Downloads changes and branches from the remote repository to the local repository, but does not automatically merge them into the current working files, allowing review before integration.

Merge (merge): Integrates changes from one branch into another. This operation often involves resolving potential conflicts that may arise when multiple developers modify the same lines of code.

Features and benefits of GIT

- Git is a free, open-source distributed version control system that efficiently manages projects of any size. Below are the key features that make Git the most popular version control system in the world

Features and benefits of GIT

Key Git Concepts

- Repository: A folder where Git tracks your project and its history.
- Clone: Make a copy of a remote repository on your computer.
- Stage: Tell Git which changes you want to save next.
- Commit: Save a snapshot of your staged changes.
- Branch: Work on different versions or features at the same time.
- Merge: Combine changes from different branches.
- Pull: Get the latest changes from a remote repository.
- Push: Send your changes to a remote repository.

Features and benefits of GIT

Working with Git

- Initialize Git on a folder, making it a Repository
- Git now creates a hidden folder to keep track of changes in that folder
- When a file is changed, added or deleted, it is considered modified
- You select the modified files you want to Stage
- The Staged files are Committed, which prompts Git to store a permanent snapshot of the files
- Git allows you to see the full history of every commit.
- You can revert back to any previous commit.
- Git does not store a separate copy of every file in every commit, but keeps track of changes made in each commit!

Features GIT

1. Open Source

Git is free and open-source, maintained by a community of developers. Anyone can contribute to its development, suggest features, or fix bugs.

- There is a large community support with plenty of tutorials, guides, and forums.
- Constant updates and improvements from contributors worldwide.
- No licensing cost; anyone can use it freely for personal or commercial projects.

Example: Popular platforms like GitHub and GitLab are built on Git and benefit from its open-source nature, creating an ecosystem for collaborative development.

Features GIT

2. Distributed System

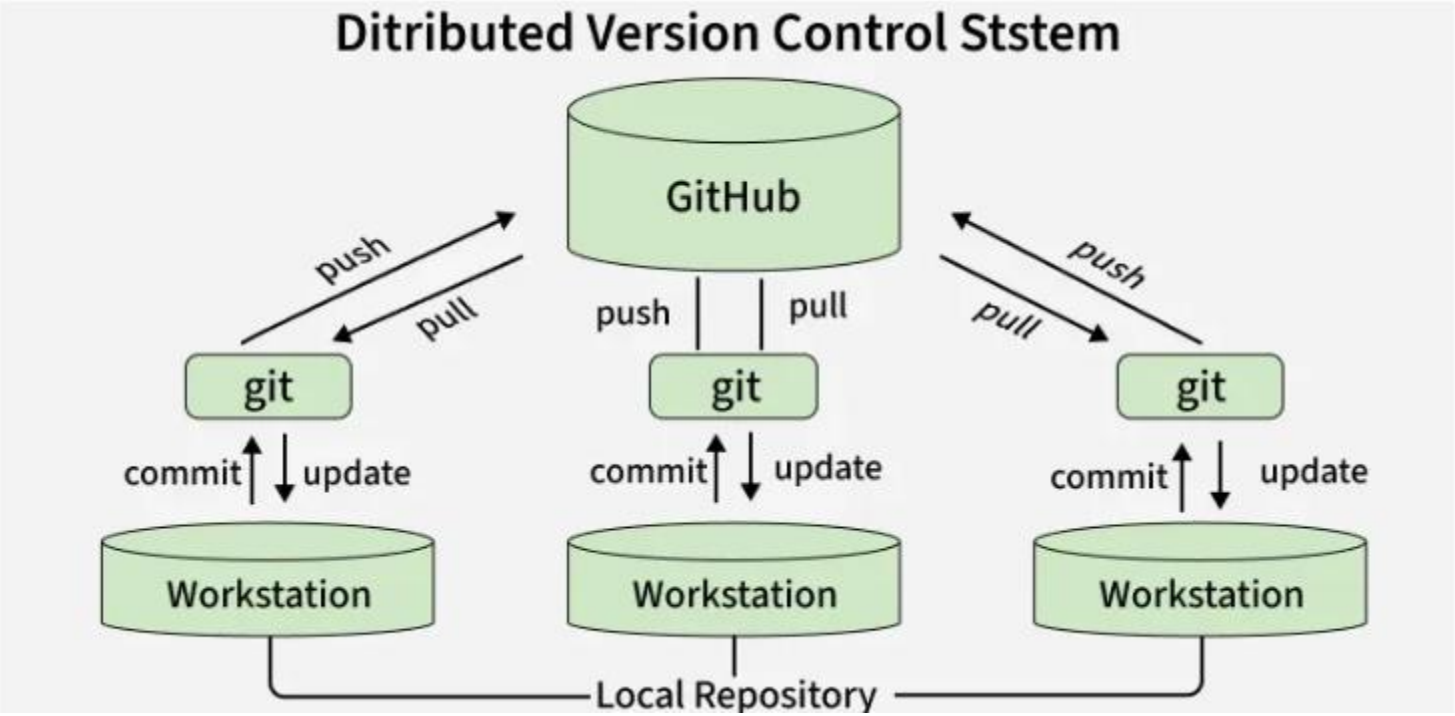
Unlike centralized version control systems, Git is fully distributed. Every developer has a complete copy of the repository, including all history, branches, and tags.

- Work offline and commit changes without a network connection.
- Redundancy ensures backups across multiple local repositories.
- Faster operations since most actions are local.
- Enables non-linear development with multiple branches.

Example: You and your teammate can both make changes to the project on your own computers and later merge your work without needing to be connected to a server.

Features GIT

2. Distributed System



Features GIT

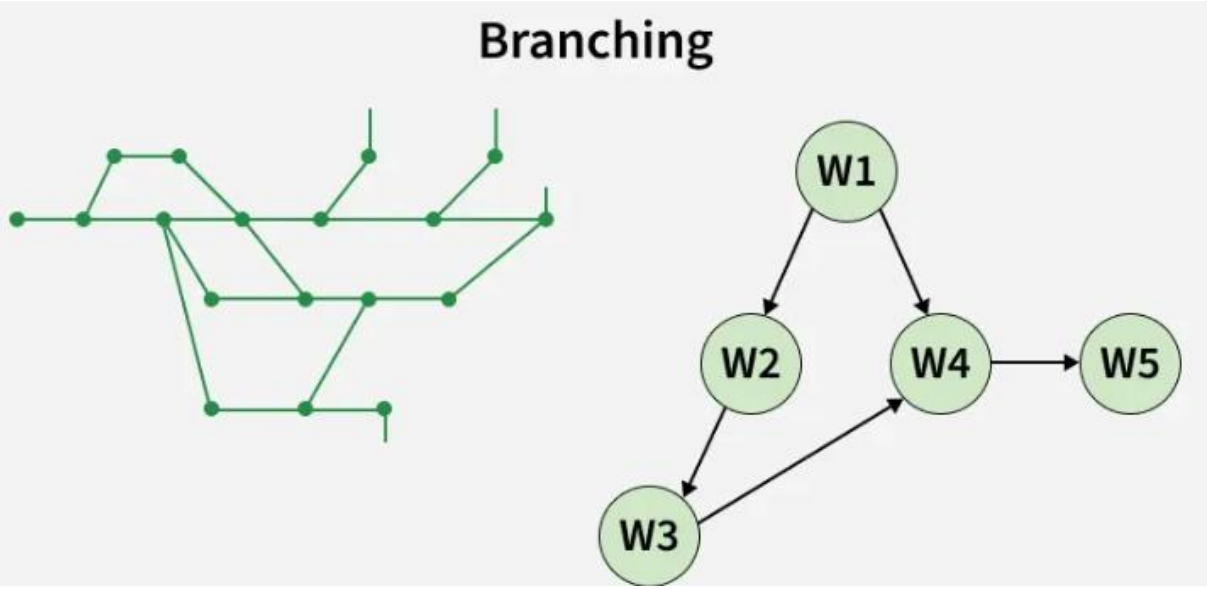
3. Branching

A branch in Git is like a parallel version of your code, allowing you to work on different tasks independently without affecting the main codebase.

- Lets you develop new features, fix bugs, or experiment safely.
- Each branch maintains its own set of commits and changes.
- You can later merge your branch back into the main branch once the work is complete.
- Helps in isolating issues, so bug fixes don’t interfere with other development work.

Features GIT

3. Branching



Types of Branches

- Main Branch: The primary stable codebase, usually main or master.
- Feature Branches: For new features, e.g., feature-login.
- Bugfix Branches: For fixing bugs, e.g., bugfix-header.
- Release Branches: For preparing a version for production.

Features GIT

3. Branching

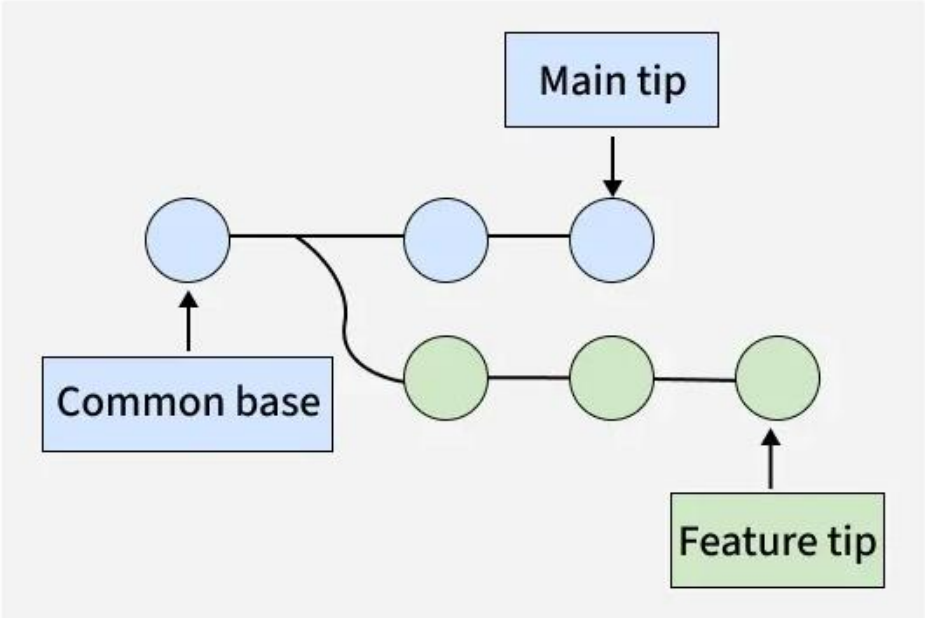
Example

If a bug is found in the main branch, you can create a separate bugfix branch, fix the issue there, test it, and then merge it back into the main branch - keeping your production code stable during the process.

Features GIT

4. Merging

- Merging is the process of combining changes from one branch into another.
- Typically, you merge a feature branch into the main branch after development is complete.
- Git tries to automatically integrate changes, but sometimes conflicts need to be resolved manually.



Features GIT

Types of Merges

1. Fast-Forward Merge:

Happens when the main branch has not moved forward since the branch was created.

Git just moves the main branch pointer forward to the latest commit.

2. Three-Way Merge:

Happens when both branches have new commits since they diverged.

Git creates a new merge commit that combines changes.

3. Merge Conflicts:

Occurs when changes on both branches affect the same line of a file.

Git cannot automatically merge and asks you to resolve conflicts manually.

Features GIT

5. History Tracking

Git keeps a complete history of all changes to the codebase. Every commit is recorded with a timestamp, author, and a message describing the change.

You can see the evolution of the project over time.

Ability to revert to a previous state if something breaks.

Helps identify bugs by checking which changes introduced an issue.

Example: If a new feature causes a bug, you can review the commit history to find exactly which changes caused the problem and fix it.

Features GIT

6. Staging Area(Index)

Git has a staging area (index) where changes can be reviewed before being committed. This allows selective commits instead of committing all changes at once.

- Enables careful review of code changes before they become part of the history.
- Helps in splitting large changes into smaller, meaningful commits.

Example: You edited three files but only want to commit two. You can stage only those two files and commit them, leaving the third file for later.

Features GIT

7. Speed

Git is highly optimized for speed. Its underlying data structures, such as the SHA-1 hash and compressed snapshots, make operations like committing, branching, and merging very fast.

- Efficient for large projects with thousands of files.
- Fast operations allow developers to experiment and iterate quickly.

Example: Creating a new branch or switching between branches takes almost no time, even for large codebases

Features GIT

8. Security

Git uses cryptographic SHA-1 hashes to ensure the integrity of the codebase. Each commit has a unique hash that represents its content and history.

- Tampering is detectable because any change in the code changes the hash.
- Ensures the authenticity of commits.

Example: If someone tries to change old commits, Git will detect that the hashes don’t match, protecting the code from unnoticed tampering.

Features GIT

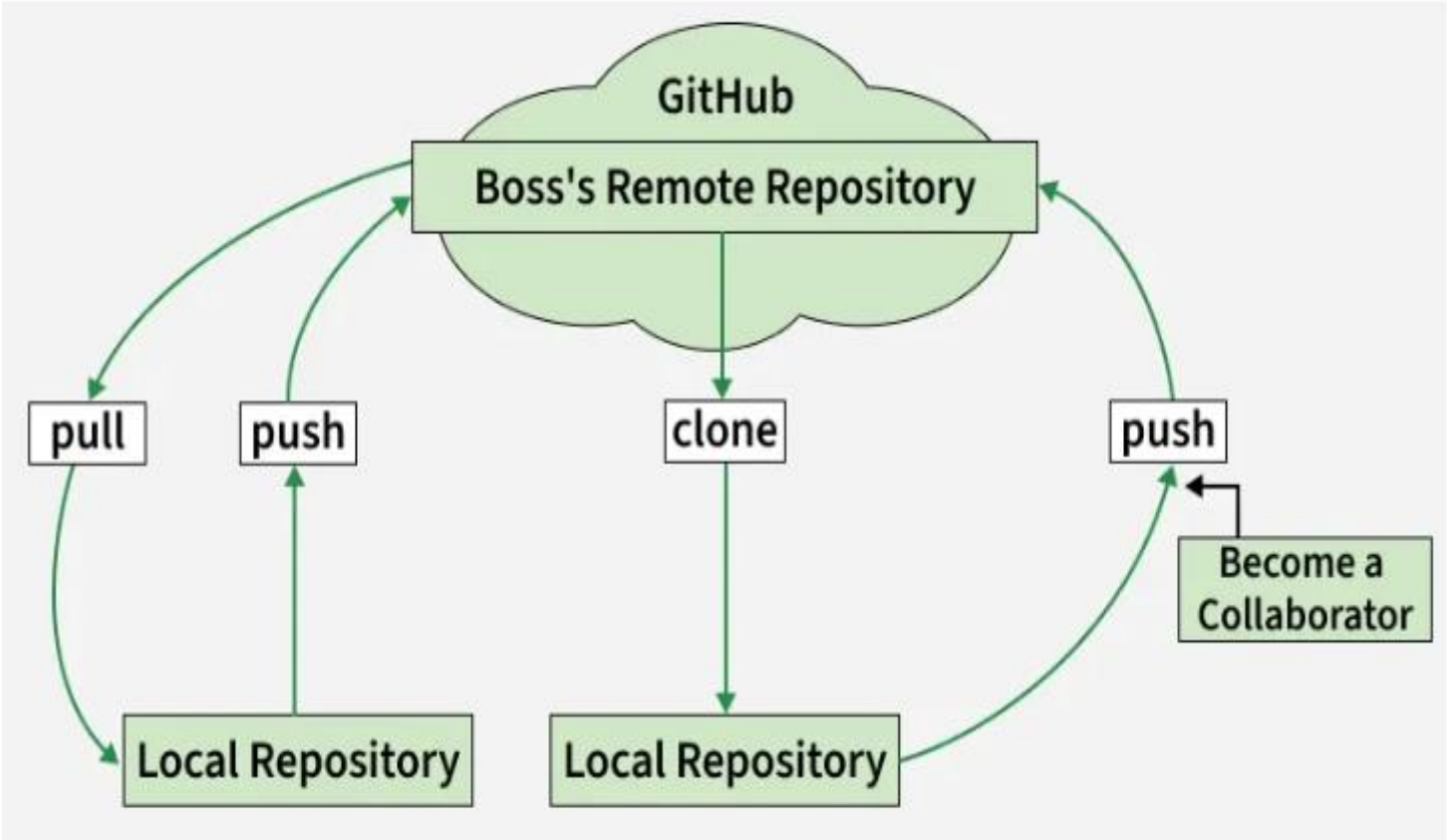
9. Collaboration

Git makes teamwork seamless by allowing multiple developers to work on the same codebase.

- Developers can clone repositories and contribute independently.
- Changes can be merged smoothly.
- Platforms like GitHub/GitLab provide pull requests and code reviews for effective collaboration.

Features GIT

9. Collaboration



Features GIT

10. Cross-Platform Support

Git works across major operating systems including Windows, Linux, and macOS.

- Provides consistent performance on all platforms.
- Wide adoption across teams with different systems.

Features GIT

11. Integration with DevOps & CI/CD Tools

Git integrates seamlessly with DevOps pipelines and CI/CD tools like Jenkins, GitHub Actions, and GitLab CI.

- Continuous Integration (CI): Every commit triggers automated builds and tests.
- Continuous Deployment (CD): Tested code can be automatically deployed to staging or production.
- Branch-based workflows: Different branches can trigger different pipelines (e.g., dev - test, main - production).
- Tool Support: Works with Jenkins, GitHub Actions, GitLab CI, CircleCI, AWS CodePipeline, etc.

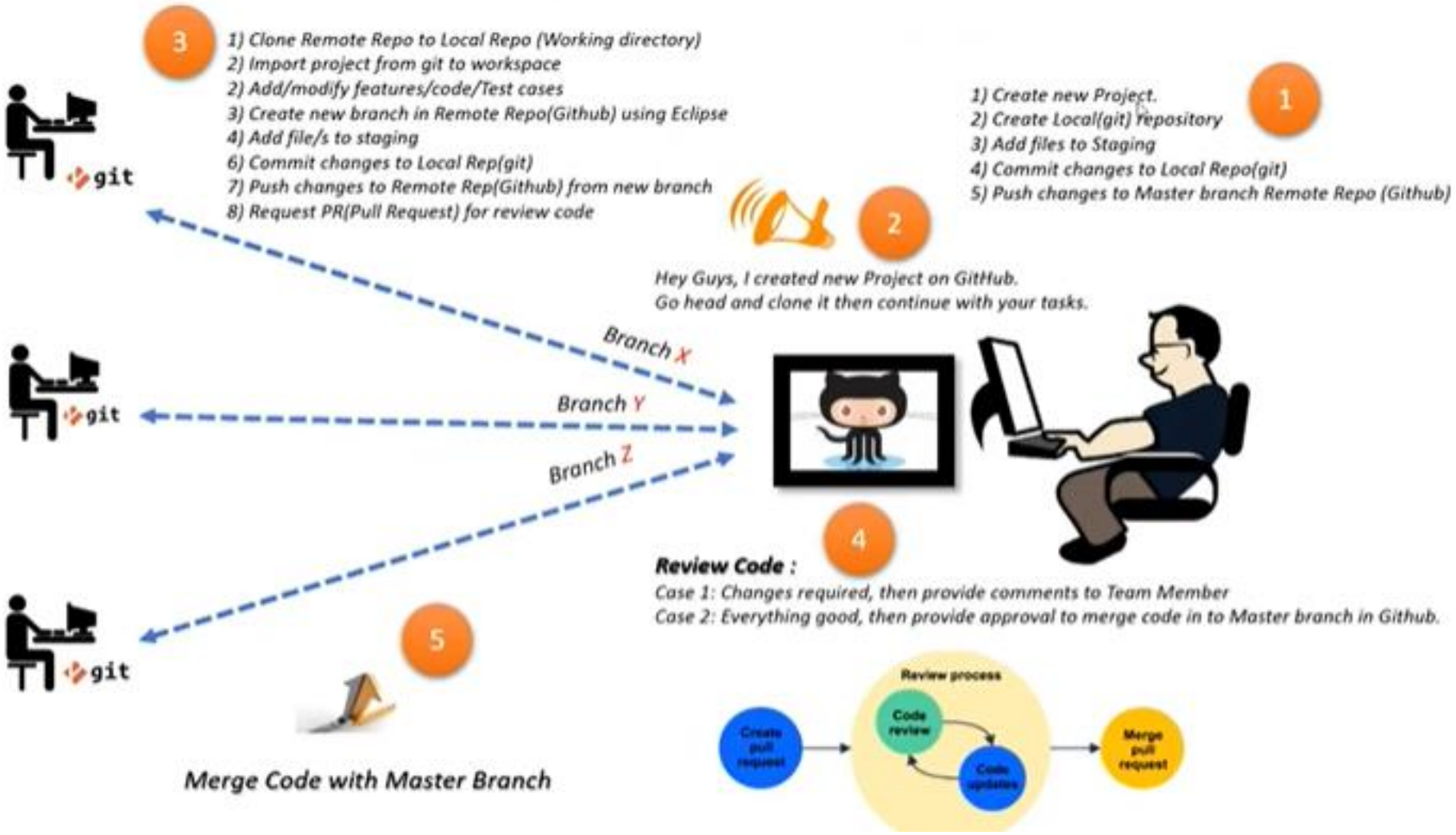


Figure from Internet